An AI-enhanced process mining framework for software process insights

Minh Khoi Nguyen, Hanh Nhi Tran, Razan Abualsaud, and Ileana Ober
Institut de Recherche en Informatique de Toulouse (IRIT)
Toulouse, France
Email: {minh-khoi.nguyen, hanh-nhi.tran, razan.abualsaud, ober}@irit.fr

2023 15th International Conference on Knowledge and Systems Engineering (KSE) | 979-8-3503-2974-2/23/$31.00 ©2023 IEEE | DOI: 10.1109/KSE59128.2023.10299475

***Abstract*—Using process mining to gain insight of software processes presents several challenges, especially in preparing good-quality input data for the mining algorithm from the low-level data recorded by non-process-aware systems, such as version control systems. This paper proposes a process mining framework integrating AI techniques to enhance and simplify the execution of process mining tasks in this context. First, we implement a learning process to build an ontology capturing the knowledge of the analyzed processes' domain. Then this domain ontology is used in constructing an event log that enables an effective process discovery and analysis. The domain ontology plays the role of a domain expert in matching low-level events to high-abstraction and meaningful process activities. We implemented and experienced the proposed framework to extract process knowledge of e-commerce web applications development.**

***Index Terms*—process mining, event log construction, domain ontology, ontology learning, process insight,**

I. Introduction

Driven by the rapid evolution of technologies and new intricate real-world problems in the big data, cloud computing, and Internet of things (IoT) era, the complexity of modern software has been steadily increasing. In this dynamic landscape, efficient and rigorous processes are crucial to ensure the success of development projects and to deliver high-quality software products. As a result, nowadays, analyzing past projects to eventually propose process improvements is a critical task in the software industry. To this end, process mining, an analytical discipline standing at the interrelation between process and data science, is a prominent solution.

Process mining (PM) focuses on extracting novel process knowledge from event data collected during past process executions [1]. A successful PM demands a well-structured event log containing event data which provide good-quality information corresponding to the traces of performed activities. In reality, event logs are mostly recorded by non-process-aware systems (e.g. ERP, CRM, message-based systems and IoT systems). Such even logs are often less structured, low-level abstracted and possibly imprecise or incomplete. Consequently, the first and most effort-demanding steps in PM are extracting and preparing event logs. Automating these time-consuming steps would be highly beneficial but challenging because they require high-level expertise in both domain knowledge and data skill to deal with heterogeneous, incomplete and/or fine-grained data [2].

Applying PM to gain insight of software development processes will face the aforementioned problem of data extraction and preparation. Collaborative software projects often use version control systems to manage their development artifacts. Such systems record the updates made during the development as event data. Because more than one updates (on one or many software artifacts) can be done when performing an activity, the history log of a version control system does not convey exactly the sequence of executed process activities. Moreover, another challenge lies in extracting shared semantics elements (e.g. activity, artifact) from various update messages when synonyms are used to label the same element, making it difficult to establish consistent meaning across different expressions.

In this setting, we propose an AI-enhanced process mining framework to improve and ease the exploitation of updates events in a version control system to obtain process knowledge. Our solution consists of (1) a semi-automatically domain ontology construction to capture software development knowledge, (2) using this ontology to filter and cluster low-level update events and (3) to associate these events to meaningful software process activities and artifacts. We provide a concrete example of applying the framework in a process mining project to discover and analyze the common process of ecommerce web applications implementation.

The rest of this paper is structured as following: Section II provides the process mining background. Section III presents our AI-enhanced process mining framework. An experiment illustrating the use of the proposed framework is described in Section IV Section V concludes the paper with some insight on the future work.

II. Process Mining

A process model describes the activities to be performed to achieve specified objectives [3]. Executing process activities leaves traces as history data stored in software systems associated to these activities. The process execution traces can be extracted and stored in an event log representing the real behavior of the actual process.

As shown in Fig. 2, process mining techniques can be used to discover a process model from an event log, to check the conformance of the actual process and its discovered process model, or to identify process problems and propose improvements.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 22:48:22 UTC from IEEE Xplore. Restrictions apply.

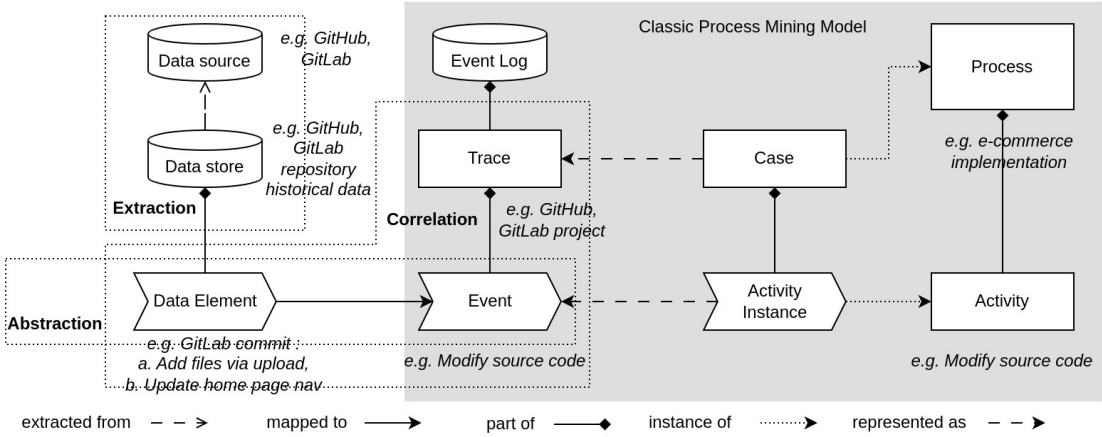

Fig. 1. Relating data and processes [4]

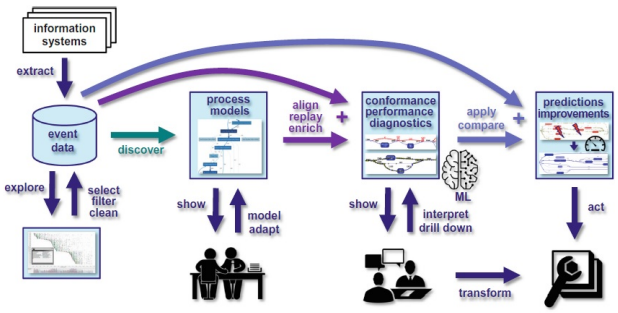

Fig. 2. Process mining landscape [5]

An event log contains a list of events generated from the executions of process activities (c.f. Fig. 1).

To allow meaningful and effective applications of traditional PM techniques, the event log must have the following properties [5]: **(1)** each event should be linked to a process instance, a.k.a. *case*, typically by using a **CaseID**, **(2)** each event should correspond to an *activity* executed within the process, by associating the event with an **Activity**, and **(3)** there is an ordering of the events pertaining to a case, derived from the **Timestamp** of the event.

These three requirements entail *CaseID*, *Activity* and *Timestamp* as the mandatory attributes of an event. An event can have additional attributes containing valuable context information, such as resource, cost, etc., which can be exploited to gain better insights into the process. Obtaining an event log that satisfies the above data requirements, however, is complicated in many real-world scenarios and a major obstacle for the adoption of PM [4].

Process model discovery is an essential step before other analysis. Alpha [6], Heuristics [7], and Fuzzy [8] are among the popular mining algorithms. The choice of an appropriate mining algorithm to use depends on many factors, such as the quality of the event log and the purpose of analysis.

III. AI-ENHANCED PROCESS MINING FRAMEWORK

In this section, we explain the motivation of the integration of a domain ontology into the process mining framework, then we present the application of AI techniques in each phase of the framework to enhance mining results.

A. Overview

Preparing high-quality event logs is crucial for obtaining accurate results with process mining [2], [5], particularly when dealing with heterogeneous, low-level, and incomplete event data recorded by non-process-aware systems. In software development, historical data extracted from version control systems (e.g. Git or Subversion) used to manage projects have the aforementioned quality issues. The data element of historical data is usually the commands made by developers, which is a *low-level* operation, e.g. the *commits* confirming the modifications of software artifacts of the projects. The characteristic of data element leads to complex relation with *high-level*, observable process activities. Some commands are shared among activities, and some complex activities yield multiple commands. For example, in Fig. 1, two low-level commits *Add files via upload* and *Update home page nav* can be mapped to the same high-level activity *Modify source code*. Furthermore, the data element is *incomplete* in that its attributes describing the high-level process activity (such as the activity name, involved artifacts) are hidden or missing. The event log preparation from historical data of version control systems is especially challenging because of the above problems. It typically involves three techniques (c.f. Fig. 1): *extraction* (obtaining relevant event data from data source), *correlation* (mapping extracted event to trace), and *abstraction* (translating low-level data element into high-level event representing process activity) [5]. Knowledge about the software development for a specific domain is strongly needed, principally to extract activity and artifact names from the data element (e.g. commit message), then classify events and map them to the corresponding activities and artifacts in the domain. We emphasize that a pure syntactical analysis of the

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 22:48:22 UTC from IEEE Xplore. Restrictions apply.

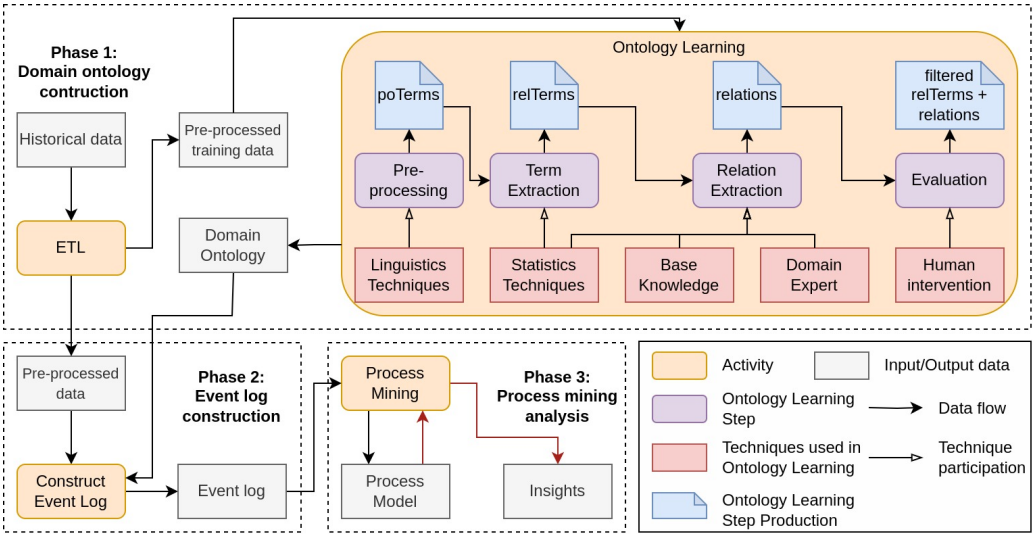

Fig. 3. An AI-enhanced process mining framework incorporating the construction of a domain ontology to enable semantics analysis

text messages in this context is very limited. It cannot handle synonyms used in different messages to refer to the same activity, and cannot establish the relations between activities. Consequently, all of those activities are seen unrelated and thus making poor results of process mining.

Manual event log preparation is very demanding in terms of time and expertise required [2] and infeasible in an online setting (i.e. with data stream) for process mining [9]. Using ontology capturing domain knowledge to assist event log preparation can help reduce the efforts and improve the effectiveness of process mining [2], [10]. Therefore, in the framework presented in Fig. 3, besides two main phases *event log construction* and *process mining analysis*, we propose a preliminary phase *domain ontology construction*,

In the first phase, we apply an ontology learning technique for extracting domain knowledge from event data and constructing a domain ontology formalizing that knowledge. The second phase uses the created domain ontology, which reflects the links between the terms in event data and the domain concepts, to reason the relation between event data and thus enable an automatic event log preparation. The pre-processed event log then can be used in the last phase to discover process models and realize analysis. We present in the next sections the implementation of each phase in the proposed framework.

B. Domain ontology construction

Within the suggested framework, we employ ontology learning to build a domain-specific ontology from text messages associated to events in software repositories. The resulting ontology encompasses concepts that are particularly relevant to the domain of processes to be analyzed.

Inspired by the work in [11], we organized the ontology learning process into the main steps, as shown in Fig. 3:

1) Pre-processing: we apply the NLP techniques to extract from the raw messages the list $poTerms$ containing the words that are potential domain terms: *verbs* are possibly activities and *nouns* are possible artifacts.

2) Term Extraction: this step extracts, from the $poTerms$, the list $relTerms$ containing relevant terms with respect to the domain concepts. This extraction can be done automatically by using either linguistic or statistical approaches. Inspired by the work in [11], we use a combination of symbolic and statistical methods based. For each term t in a domain k, a score is calculated based on the domain relevance and consensus of the term in several documents of the target domain. Terms having the final score above a predefined threshold are kept as relevant terms of the domain.

3) Relation Extraction: this step establishes the relations between the terms obtained in $relTerms$. We focus on extracting the relations between activities and artifacts of the domain so that the domain ontology could help in event data abstraction and correlation [12]. Two knowledge sources are used to extract different kinds of relations :

- *Base knowledge* such as lexical databases (e.g. *WordNet*, *SemCor*) or dictionaries (e.g. $Cambridge$ or $Oxford$) provides a trustful information source to extract the ontological relations (e.g., taxonomy, equivalence, etc.) between terms [11].
- *Knowledge of domain experts* provides information about domain relations between a specific domain's concepts, which is crucial to establish relations between the terms in the ontology. In our framework, the domain expert's knowledge is given in the form of a metamodel defining the main concepts of software development and the relations between them (*gray classes* in Fig.4). The generic concepts at the top level are refined at lower levels into more specific concepts of software development process. For example, an *Activity* can be *Manage* or *Develop*. An *Artifact* can be specialized further by their role in the development of a particular domain application.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 22:48:22 UTC from IEEE Xplore. Restrictions apply.

For example, a *Software Artifact* can be a *View* or a *Component* or, more specific, an *Ecommerce component*.

The metamodel capturing the expert's knowledge defines the multiple-levels structure of the domain ontology, which will be enriched with the concepts corresponding to the extracted terms in the $relTerms$ list (*orange classes* in Fig. 4). By consulting the base knowledge, the ontology learning knows the **definition** and the **synonyms** of an extracted term t and thus can create the relations *is-a* between t and the concepts in higher levels as well as the relations *hasSynonym* between t and other terms. For example, the extracted term *implement* is a $Develop$ activity, and it has *code* as a synonym.

4) Evaluation: At this stage, we need an expert for the validation of the domain ontology before using it.

C. Event log construction

The objective of this phase is creating the final event log satisfying the requirements presented in Section II: each event should have a *Timestamp* associated to an *Activity* and should refer to a *CaseID*. While the timestamp can be easily found from the event's metadata, the activity's name and the *CaseID*, which is the identification of the repository, must also be extracted from the event's message.

There are two difficulties that we need to overcome in the correlation and abstraction of event data:

Convergence and Divergence: Due to potential variations in the definitions of case notions, an event log may encounter *convergence* (one event is related to multiple cases) and *divergence* (independent, multiple executions of a group of activities within a single case) [13]. Neglecting these problems when performing process mining may result in inaccurate process models.

Inspired by the approach in [13], to avoid convergence and divergence problems, we build each artifact type its own event log and thus eliminate the confusion of case notion. Process discovery is performed first at the artifact-level to establish the life-cycle of each artifact, then we combine the process models of all artifacts to create the overall process model.

Synonymy of activity and artifact names: Event log can encounter the problem of synonymy, i.e. developers use different terms in their command's message to refer to the same semantics element (artifact or activity). If the same artifact is named differently in different projects, it will be considered as unrelated artifacts whose event log consists only events of one process instance, which is meaningless for process discovery. As we realize process discovery for each artifact type, the problem of synonymy of artifact name may lead to low quality process models. To deal with the synonymy problem, we use the domain ontology constructed in the previous phase to detect the semantics concept of different lexical terms and thus allow a better clustering of event data.

Alg. 1 shows the construction of the final event log from raw event data by selecting only the event that can be associated to a domain activity for manipulating a domain artifact.

For each raw event, we first apply NLP technique to detect the *verb* and *noun* appeared in the event's message. Then, from the domain ontology, thanks to the relation *hasSynonym*, we retrieve the preferred terms used to refer to the *Activity* (action) and the *Artifact* (object) corresponding to the given verb and noun in the raw event's name. The filtered event will be named with *action* + *object*. We set the filtered name as 'unknown', if the retrieval is failed for both cases, such that we can (1) identify the unrecognized term and its origin to update the domain ontology if necessary, and (2) identify the abnormal activity in the process execution. The same processing is used on the raw artifact name to merge the event logs of the synonyms of an artifact.

Algorithm 1: EVENT LOG CONSTRUCTION

```
Input: preProcessedData, domainOntology
Output: filteredEventLog
/* create empty event log for each artifact                    */
artifactEventLog := emptyDict()
forall rawEvent in preProcessedData do
    filteredEvent := rawEvent
    /* detect verb and noun from raw event name                 */
    verb, noun := detectTerm(rawEvent.eventName)
    action := domainOntology.getPreferredActivity(verb)
    object := domainOntology.getPreferredArtifact(noun)
    /* mark the event with no activity and no artifact
       as 'unknown'                                             */
    if action not null and object not null then
        filteredEvent.setEventName(action + object)
    else
        filteredEvent.setEventName('unknown')
    end
    forall rawArtifact in rawEvent.getArtifacts() do
        artiVerb, artiNoun := detectTerm(rawArtifact)
        artiAction := domainOntology.getPreferredActivity(artiVerb)
        artiObject := domainOntology.getPreferredArtifact(artiVerb)
        if action not null and object not null then
            artifactEventLog.get(artiAction + artiObject).add(filteredEvent)
        else
            artifactEventLog.get('unknown').add(filteredEvent)
        end
    end
end
```

D. Process Mining analysis

After having a good-quality event log, we can select an appropriate algorithm to realize the process mining.

In this work, we apply the heuristic miner on each artifact's event log to discover its life-cycle. We explore the common points between these life-cycles, i.e. the shared activities involving the artifacts, as the joints to connect the life-cycles. The discovered process model, can be used for further analysis or as a starting point for process automation.

IV. EXPERIMENT

A. Set up

We applied our AI-enhanced framework to conduct a process mining project for getting insights of processes for implementing e-commerce applications. Through this case study, we investigated the role of the integrated domain ontology in the success of the process mining project.

1) Historical data: We used the historical data of the real software projects archived in Software Heritage[1] The project repositories originate from various repository host: GitHub,

[1]https://www.softwareheritage.org

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 22:48:22 UTC from IEEE Xplore. Restrictions apply.

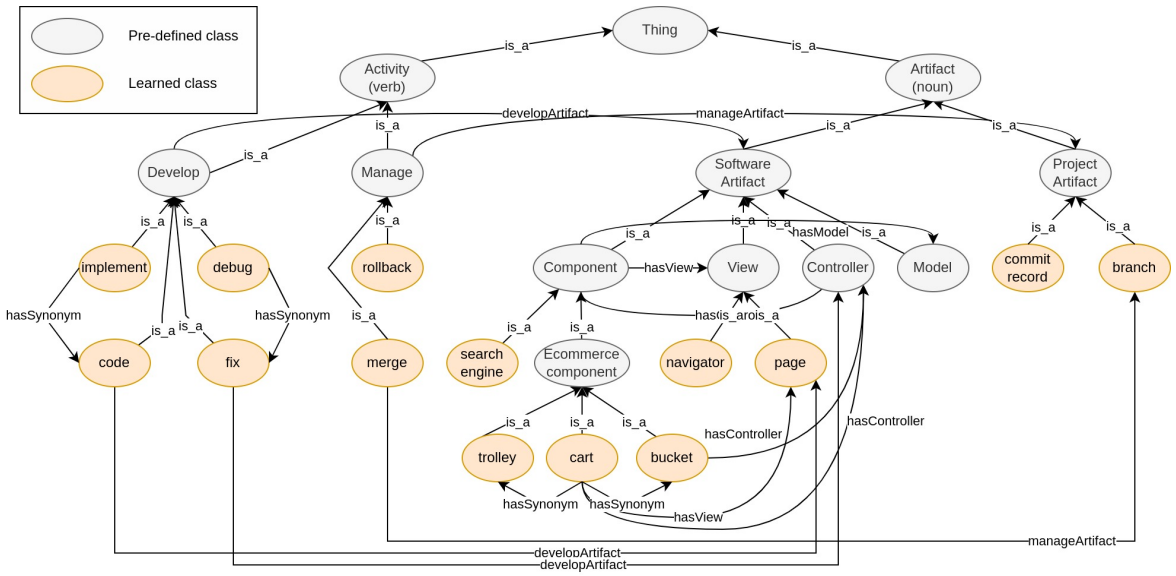

Fig. 4. Ecommerce Implementation Process ontology

GitLab, BitBucket, etc. We extracted 200 ecommerce projects from GitHub for the process mining, among them, 100 projects are used as the training dataset for the ontology learning. To compute the statistics measure in the term extraction step, we also extracted data from projects of two other domains, Loan Application [14] and Road Traffic Fine Management [15].

2) Ontology learning: The linguistics techniques applied in the pre-processing step of the ontology learning is provided by Java API OpenNLP 1.8.4[2]. For the base knowledge, we used the service of Thesaurus.com.

B. Result

1) Performing process mining without domain ontology to improve the raw event log: in this set up, the event log contains many unrelated events because of the synonymy in event names. Consequently, these events have low frequencies, i.e. low possibility of reappearance in the event log, and thus cannot provide useful information to allow establishing relations between activities in the process model. The Heuristics Miner, which heavily relies on the dependencies and frequencies between the activities, eliminates those low-frequencies activities thus **cannot construct a meaningful process model eventually**.

2) Applying our AI-Enhanced framework integrating the domain ontology to improve the event log quality: thanks to the use of the domain ontology to rename events and to recognize synonyms, the event log contains less unrelated event. Consequently, the process mining could discover the process model in Fig.5. The image (b) shows the whole process model and the image (a) shows its extract. This ”spaghetti-like” model is an accurate reflection of the real unstructured executions of the process. This result claims the importance of the domain ontology in the framework to recognize the share semantics between the activities, thus contribute to the success of process mining.

The discovered model provides information about: (1) the number of times that each activity occurred, and (2) the flow of the activities. Using this information, we could analyze further the processes to get insights into :

(P1) Time consumption: the average time needed to finish an activity as shown in Table I. Because GitHub only records the completion events of activities, we had to assume that the starting time of an activity is the completion time of its preceding activity. This assumption leads to the result NaN for starting activities. Time analysis helps discover bottlenecks at some particular points, and identify the most time-demanding activities in the process.

TABLE I
TIME ANALYSIS OF ECOMMERCE DEVELOPMENT ACTIVITIES

Activity name	Average time
filter attribute	NaN
page product	4647734.47
...	...
page order	1026285.67
...	...
start branch	43

(P2) Rewinds: intentional and unintentional rewinds of tasks, which could be respectively an effective way to perform the process or an anomaly execution.

(P3) Variants: probability of the branching after each activity, due to a decision or coordination of simultaneous activities.

Fig. 5 displays the results of Rewind (red arrows) and Variant (percentage besides each branch) analysis on the discovered process model. We observe the omnipresence of the rewinds, especially to the destination of the **unknown** activity. This result reflects the limited coverage capability of the learned domain ontology: it cannot recognize the irrelevant term to the domain, or a new word not appearing in the training

[2]https://opennlp.apache.org

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 22:48:22 UTC from IEEE Xplore. Restrictions apply.

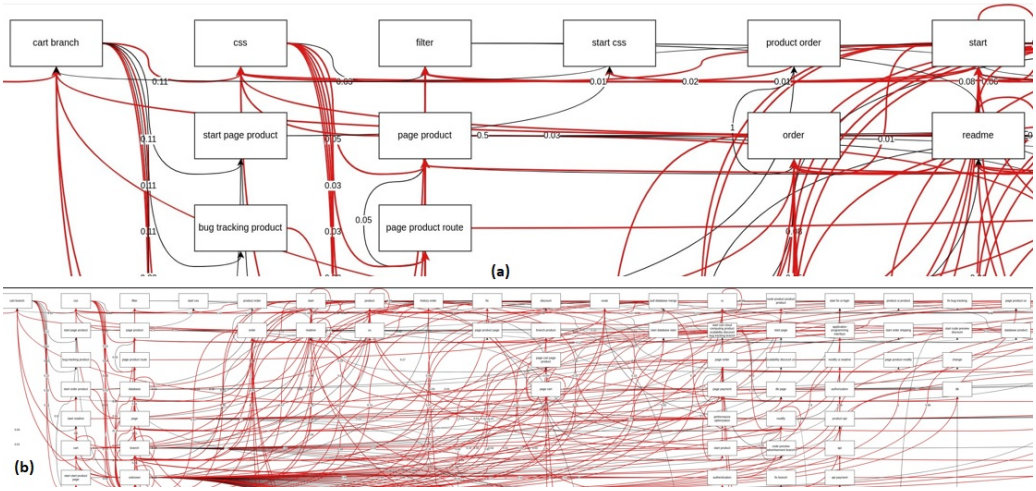

Fig. 5. Extract of the Ecommerce Implementation process model discovered from Software Heritage data

projects. These undetected terms are noted in a mapping file to update the domain ontology if necessary. Moreover, the Variant analysis provides us information about the most probable scenarios of process executions.

V. Conclusion

This paper presents a process mining framework integrating a semi-supervised learning to construct a domain-ontology from the raw data and to use it as a knowledge source to improve the quality of the event log. The proposed framework defines a generic structure and the main phases, together with suggested AI-methods to assist the process mining tasks. It could be customized when applying to a specific mining project, by principally considering:

- *Application domain*: The characteristics of historical data vary from one application domain to another and depend directly on particular management tools used in each domain. Consequently, when applying the proposed framework to a specific domain, it is unavoidable to reconsider the AI techniques to be employed to extract event data. Changing the domain application might also impact the ontology learning approach according to the availability of training data, base knowledge and experts in the domain.
- *Mining purpose*: the wished process analysis determines the mining algorithm to use and the required event's attributes, as well as the expert knowledge to provide. Changing the analysis thus impacts directly the event log preparation steps, and impacts indirectly the ontology learning.

This preliminary work reveals new challenges, especially on ontology learning. As a future work, we envisage to evolve the framework by enriching the base knowledge with understandings obtained from the discovered process model and its analysis. We are also studying the possibility of using a reinforcement learning to integrate better domain experts in constructing a domain ontology.

References

[1] W. M. P. van der Aalst, *Process Mining: Data Science in Action*. Heidelberg: Springer, 2 ed., 2016.

[2] F. Folino and L. Pontieri, "Ai-empowered process mining for complex application scenarios: Survey and discussion," *J. Data Semant.*, vol. 10, no. 1-2, pp. 77–106, 2021.

[3] R. Lee and B. Dale, "Business process management: a review and evaluation," *Business Process Management Journal*, vol. 4, no. 3, pp. 214–225, 1998. Publisher: MCB UP Ltd.

[4] K. Diba, K. Batoulis, M. Weidlich, and M. Weske, "Extraction, correlation, and abstraction of event data for process mining," *WIREs Data Mining and Knowledge Discovery*, vol. 10, no. 3, p. e1346, 2020.

[5] W. van der Aalst and J. Carmona, *Process Mining Handbook*. Lecture Notes in Business Information Processing, Springer International Publishing, 2022.

[6] W. van der Aalst, T. Weijters, and L. Maruster, "Workflow mining: discovering process models from event logs," *IEEE Transactions on Knowledge and Data Engineering*, vol. 16, no. 9, pp. 1128–1142, 2004.

[7] A. Weijters and J. Ribeiro, "Flexible heuristics miner (FHM)," in *2011 IEEE Symposium on Computational Intelligence and Data Mining (CIDM)*, pp. 310–317, IEEE, 2011.

[8] C. W. Günther and W. M. P. van der Aalst, "Fuzzy mining – adaptive process simplification based on multi-perspective metrics," in *Business Process Management* (G. Alonso, P. Dadam, and M. Rosemann, eds.), vol. 4714, pp. 328–343, Springer Berlin Heidelberg, 2007. Series Title: Lecture Notes in Computer Science.

[9] I. M. Helal and A. Awad, "Online correlation for unlabeled process events: A flexible cep-based approach," *Information Systems*, vol. 108, p. 102031, 2022.

[10] A. Medeiros, W. Aalst, and C. Pedrinaci, "Semantic Process Mining Tools: Core Building Blocks," in *16th European Conference on Information Systems (ECIS'08)* (W. Golden, T. Acton, K. Conboy, H. Heijden, and V. Tuunainen, eds.), (Galway, Ireland), pp. 1953–1964, 2008.

[11] R. Navigli and P. Velardi, "Learning domain ontologies from document warehouses and dedicated web sites," *Computational Linguistics*, vol. 30, no. 2, pp. 151–179, 2004.

[12] H. A. Reijers, J. Mendling, and J. Recker, "Business process quality management," in *Handbook on Business Process Management*, 2015.

[13] W. M. P. van der Aalst, "Object-centric process mining: Dealing with divergence and convergence in event data," in *Software Engineering and Formal Methods: 17th International Conference, SEFM 2019, Oslo, Norway, September 18–20, 2019, Proceedings*, pp. 3–25, Springer-Verlag, 2019.

[14] B. van Dongen, "Loan process: Bpi challenge 2017," 2017. https://data.4tu.nl/articles/_/12696884/1.

[15] M. M. de Leoni and F. Mannhardt, "Road traffic fine management process," 2015. https://data.4tu.nl/articles/_/12683249/1.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 22:48:22 UTC from IEEE Xplore. Restrictions apply.